

Tworzenie aplikacji webowych

Zaawansowane funkcje i wdrożenie

STUDIA PODYPLOMOWE

Data Science w zastosowaniach biznesowych

Uniwersytet Warszawski, Wydział Nauk Ekonomicznych



UNIwersYTET
WARSAWski



WYDZIAŁ NAUK
EKONOMICZNYCH

Ewa Weychert, PhD

Copyright © 2025 WNE UW. All rights reserved.

Spis Treści

- 1 Zaawansowane widgety
- 2 Zaawansowane opcje layoutu
- 3 Wdrożenie aplikacji
- 4 GitHub i Streamlit Cloud
- 5 Wdrożenie na Streamlit Cloud — krok po kroku
- 6 Dobre praktyki

Czego nauczyliśmy się do tej pory

- Podstawowe widgety: suwak, pole tekstowe, checkbox, przycisk.
- Wczytywanie CSV i podstawowe EDA.
- Organizacja aplikacji: pasek boczny, zakładki, kolumny.
- Teraz idziemy dalej:
 - bardziej zaawansowane widgety,
 - zaawansowane opcje layoutu,
 - wdrożenie aplikacji w internecie.

st.expander — zwijane sekcje

```
import streamlit as st

with st.expander("Kliknij, aby zobaczyć szczegóły"):
    st.write("To jest ukryta treść.")
    st.write("Można tu umieścić tabele, wykresy itp.")
```

- st.expander tworzy sekcję, którą użytkownik może rozwinąć lub zwinąć.
- Przydatny do:
 - ukrywania długich opisów metodologii,
 - opcjonalnych podglądów danych,
 - dokumentacji wewnątrz aplikacji.

Uruchom app_exp_1.py

```

app_exp_1.py
1  import streamlit as st
2
3  st.title("Przykład: st.expander")
4
5  with st.expander("Kliknij, aby zobaczyć szczegóły"):
6      st.write("To jest ukryta treść.")
7      st.write("Można tu umieścić tabele, wykresy itp.")
8
9  with st.expander("Metodologia"):
10     st.write("Dane pochodzą z bazy GUS za lata 2020-2024.")
11     st.write("Zastosowano regresję liniową.")
12
13  with st.expander("Słownik pojęć"):
14     st.write("==EDA== – Exploratory Data Analysis (eksploracyjna analiza danych)")
15     st.write("==KPI== – Key Performance Indicator (kluczowy wskaźnik efektywności)")
16

```

Przykład: st.expander

▸ Kliknij, aby zobaczyć szczegóły

▸ Metodologia

▸ Słownik pojęć

Przykład: st.expander

▾ Kliknij, aby zobaczyć szczegóły

To jest ukryta treść.

Można tu umieścić tabele, wykresy itp.

▾ Metodologia

Dane pochodzą z bazy GUS za lata 2020-2024.

Zastosowano regresję liniową.

▾ Słownik pojęć

EDA – Exploratory Data Analysis (eksploracyjna analiza danych)

KPI – Key Performance Indicator (kluczowy wskaźnik efektywności)

st.metric — wskaźniki KPI

```
import streamlit as st

st.metric(label="Przychód", value="120 000 zł",
          delta="+ 8%")
st.metric(label="Liczba klientów", value=340,
          delta=-12)
```

- `st.metric` wyświetla wartość z etykietą i zmianą (delta).
- Delta zielona = wzrost, czerwona = spadek.
- Idealny do dashboardów biznesowych i raportów KPI.

Uruchom app_metric_2.py

```

Welcome  app_metric_2.py x
app_metric_2.py > ...
1  import streamlit as st
2
3  st.title("Dashboard KPI")
4
5  col1, col2, col3 = st.columns(3)
6
7  with col1:
8      st.metric(label="Przychód",
9                value="120 000 zł",
10               delta="+8%")
11  with col2:
12      st.metric(label="Liczba klientów",
13                value=340,
14                delta=-12)
15  with col3:
16      st.metric(label="Zwroty",
17                value="4%",
18                delta="-1%")
19
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
ewaweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_exp_1.py
ewaweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_metric_2.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.168:8501

For better performance, install the Watchdog module:

$ xcode-select --install
$ pip install watchdog

```

Dashboard KPI

Przychód

120 000 zł

↑ +8%

Liczba klientów

340

↓ -12

Zwroty

4%

↓ -1%

st.progress i st.spinner

```
import streamlit as st
import time

# Pasek postępu
bar = st.progress(0)
for i in range(100):
    time.sleep(0.01)
    bar.progress(i + 1)

# Spinner podczas ładowania
with st.spinner("Wczytuję dane..."):
    time.sleep(2)
st.success("Gotowe!")
```

- st.progress — animowany pasek postępu.
- st.spinner — kręcące się kółko podczas operacji.
- st.success / st.error / st.warning / st.info — kolorowe komunikaty.

Uruchom app_progress_3.py

```

app_progress_3.py > ...
1  import streamlit as st
2  import time
3
4  st.title("Przykład: postęp i spinner")
5
6  # Pasek postępu
7  st.subheader("Pasek postępu")
8  st.write("Kliknij przycisk, aby zobaczyć pasek postępu.")
9
10 if st.button("Uruchom przetwarzanie"):
11     st.write("Przetwarzam dane...")
12     bar = st.progress(0)
13     for i in range(100):
14         time.sleep(0.02)
15         bar.progress(i + 1)
16     st.success("Przetwarzanie zakończone!")
17
18 # Spinner
19 st.subheader("Spinner")
20 if st.button("Wczytaj raport"):
21     with st.spinner("Wczytuje dane..."):
22         time.sleep(2)
23     st.success("Raport gotowy!")
24     st.info("Znaleziono 1 240 rekordow.")
25     st.warning("Uwaga: 3 rekordy maja brakujace wartosci.")
26     st.error("Bład: brak dostępu do serwera (przykład).")
27

```

Przykład: postęp i spinner

Pasek postępu

Kliknij przycisk, aby zobaczyć pasek postępu.

Uruchom przetwarzanie

Spinner

Wczytaj raport

Raport gotowy!

Znaleziono 1 240 rekordow.

Uwaga: 3 rekordy maja brakujace wartosci.

Bład: brak dostępu do serwera (przykład).

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
<pre> ewaweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_netric_2.py ewaweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_progress_3.py You can now view your Streamlit app in your browser. Local URL: http://localhost:8501 Network URL: http://192.168.1.168:8501 For better performance, install the Watchdog module: \$ xcode-select --install \$ pip install watchdog </pre>				

st.form — formularz z przyciskiem zatwierdzenia

```
import streamlit as st

with st.form("moj_formularz"):
    imie = st.text_input("Imię")
    wiek = st.number_input("Wiek", min_value=0,
                           max_value=120, value=25)
    zatwierdz = st.form_submit_button("Zatwierdź")

if zatwierdz:
    st.write(f"Witaj, {imie}! Masz {wiek} lat.")
```

- st.form grupuje widgety — skrypt odpala się dopiero po kliknięciu przycisku zatwierdzenia.
- Bez formularza każda zmiana widgetu powoduje ponowne uruchomienie.
- Przydatne gdy użytkownik musi wypełnić kilka pól naraz.

Uruchom app_form_4.py

```

app_form_4.py > ...
1  import streamlit as st
2
3  st.title("Rejestracja uczestnika")
4
5  with st.form("formularz_rejestracji"):
6      imie = st.text_input("Imie i nazwisko")
7      wiek = st.number_input("Wiek",
8                             min_value=18, max_value=99, value=25)
9      miasto = st.selectbox("Miasto",
10                           ["Warszawa", "Krakow", "Gdansk",
11                            "Wroclaw", "Poznan"])
12      kurs = st.multiselect("Wybrane kursy:",
13                            ["Python", "SQL", "Streamlit",
14                             "Machine Learning"])
15      zgoda = st.checkbox("Akceptuje regulamin")
16      wyslij = st.form_submit_button("Zarejestruj")
17
18  if wyslij:
19      if not zgoda:
20          st.error("Musisz zaakceptowac regulamin.")
21      elif not imie:
22          st.error("Podaj imie i nazwisko.")
23      else:
24          st.success(
25              f"Zarejestrowano: {imie}, {wiek} lat, {miasto}")
26          st.write("Wybrane kursy:", kurs)
27

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

ewaweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_progress_3.py
ewaweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_form_4.py

```

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>
 Network URL: <http://192.168.1.168:8501>

For better performance, install the Watchdog module:

```

$ xcode-select --install
$ pip install watchdog

```

Rejestracja uczestnika

Imie i nazwisko

Wiek

25

- +

Miasto

Warszawa

▼

Wybrane kursy:

Choose options

▼

☐ Akceptuje regulamin

Zarejestruj

st.session_state — pamięć między uruchomieniami

Problem: Streamlit uruchamia cały skrypt od nowa przy każdej interakcji użytkownika. Zwykłe zmienne Pythona są wtedy kasowane.

Rozwiązanie: `st.session_state` — specjalny słownik, który Streamlit zapamiętuje między uruchomieniami skryptu.

```
import streamlit as st
if "licznik" not in st.session_state:
    st.session_state["licznik"] = 0
if st.button("Kliknij mnie"):
    st.session_state["licznik"] += 1
st.write("Liczba kliknięć:", st.session_state["licznik"])
```

Co robi każda linia:

- `if "licznik" not in st.session_state:`
Sprawdza, czy zmienna już istnieje. Bez tego przy każdym kliknięciu licznik resetowałby się do 0.
- `st.session_state["licznik"] = 0`
Tworzy zmienną tylko raz — przy pierwszym uruchomieniu.
- `st.session_state["licznik"] += 1`
Dodaje 1 do zapamiętanej wartości — nie do nowej zmiennej.
- `st.write(...)`
Wyświetla aktualną wartość licznika.

Kiedy to jest potrzebne?

- Liczniki kliknięć i akumulatory wartości (zmienna, która zbiera i sumuje kolejne wartości).
- Wieloetapowe formularze (krok 1 → krok 2 → krok 3).
- Logowanie użytkownika (zapamiętaj, kto jest zalogowany).
- Koszyk zakupów (zapamiętaj wybrane produkty).

Uruchom app_session_5.py

```

app_session_5.py > ...
1  import streamlit as st
2
3  st.title("Koszyk zakupow")
4
5  # Inicjalizacja stanu
6  if "koszyk" not in st.session_state:
7      st.session_state["koszyk"] = []
8
9  if "licznik" not in st.session_state:
10     st.session_state["licznik"] = 0
11
12 # Dodawanie produktow
13 st.subheader("Sklep")
14 produkt = st.selectbox("Wybierz produkt:",
15                        ["Jabliko - 2 zł", "Banan - 1.5 zł",
16                        "Pomarancza - 3 zł", "Winogrona - 8 zł"])
17
18 col1, col2 = st.columns(2)
19
20 with col1:
21     if st.button("Dodaj do koszyka"):
22         st.session_state["koszyk"].append(produkt)
23         st.session_state["licznik"] += 1
24         st.success(f"Dodano: {produkt}")
25
26 with col2:
27     if st.button("Wyczyszc koszyk"):
28         st.session_state["koszyk"] = []
29         st.session_state["licznik"] = 0
30         st.info("Koszyk wyczyszczony.")
31

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

ewaweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_progress_3.py
ewaweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_session_5.py

```

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>
 Network URL: <http://192.168.1.168:8501>

For better performance, install the Watchdog module:

```

$ xcode-select --install
$ pip install watchdog

```

Koszyk zakupow

Sklep

Wybierz produkt:

Jabliko - 2 zł

Dodaj do koszyka

Wyczyszc koszyk

Dodano: Jabliko - 2 zł

Twoj koszyk

Liczba produktow

6

1. Jabliko - 2 zł
2. Jabliko - 2 zł
3. Jabliko - 2 zł
4. Jabliko - 2 zł
5. Jabliko - 2 zł
6. Jabliko - 2 zł

Czym jest @ — dekorator?

Dekorator to sposób na „owinięcie” funkcji dodatkową funkcjonalnością — bez zmieniania jej kodu w środku.

Analogia z życia:

- Masz kanapkę (funkcja).
- Wkładasz ją do pudełka termicznego @cieplo
→ kanapka nadal jest kanapką, ale teraz jest ciepła.
- Wkładasz ją do folii @swiezosc
→ kanapka nadal jest kanapką, ale teraz jest świeża.
- Kanapka się nie zmieniła — zmieniło się jej „opakowanie”.

W Pythonie — bez dekoratora:

```
def przywitaj():
    return "Cześć!"
```

```
print(przywitaj())    # Cześć!
```

Z dekoratorem @ — funkcja dostaje nowe możliwości:

```
@st.cache_data        # ← to jest dekorator
def wczytaj_dane():
    return pd.read_csv("dane.csv")
```

- @st.cache_data mówi Pythonowi:
„zanim uruchomisz tę funkcję, sprawdź najpierw czy wynik już masz zapamiętany”.
- Funkcja wczytaj_dane sama w sobie się nie zmienia — dekorator tylko dodaje mechanizm zapamiętywania.

Czym jest cache (buforowanie)?

Cache (czyt. „kesz”) = **schowek na wyniki**.

Zamiast liczyć coś od nowa za każdym razem — zapamiętaj wynik i przy kolejnym pytaniu po prostu go oddaj.

Analogia z życia:

- Nauczyciel pyta: „ile to 237×48 ?”
- Liczysz na kartce — zajmuje to 10 sekund.
- Zapisujesz wynik na karteczce i przyklejasz na biurku.
- Nauczyciel pyta znowu: „ile to 237×48 ?”
- Patrzysz na karteczkę — odpowiadasz w 0 sekund.
- **Karteczka = cache.**

W Streamlit bez cache:

Przy każdym kliknięciu użytkownika:

```
df = pd.read_csv("duzy_plik.csv") # czeka 5 sekund...
```

W Streamlit z cache:

```
@st.cache_data
def wczytaj():
    return pd.read_csv("duzy_plik.csv")

df = wczytaj()
# Pierwsze wywołanie: czeka 5 sekund
# Każde kolejne:      natychmiastowe (z pamięci)
```

st.cache_data — buforowanie danych

Jak to działa krok po kroku:

```
import streamlit as st
import pandas as pd

@st.cache_data                                # krok 1: owijamy funkcję
def wczytaj_dane(sciezka):                    # krok 2: definiujemy funkcję
    return pd.read_csv(sciezka)              # krok 3: wczytujemy plik

df = wczytaj_dane("dane.csv")                # krok 4: wywołujemy funkcję
st.dataframe(df)                             # krok 5: wyświetlamy dane
```

Co robi każda część:

- `@st.cache_data`
Dekorator — mówi Streamlit: „zapamiętaj wynik tej funkcji”.
- `def wczytaj_dane(sciezka):`
Zwykła funkcja Pythona — wczytuje plik CSV z podanej ścieżki.
- `return pd.read_csv(sciezka)`
Odczytuje plik i zwraca DataFrame — ale tylko raz! Potem Streamlit oddaje zapamiętany wynik.
- `df = wczytaj_dane("dane.csv")`
Przy pierwszym uruchomieniu: wczytuje plik.
Przy kolejnych: natychmiastowo zwraca zapamiętany DataFrame.

Dwa rodzaje cache w Streamlit:

- `@st.cache_data` — dla danych (DataFrame, listy, liczby).
- `@st.cache_resource` — dla zasobów (połączenia z bazą danych, modele ML).

Uruchom app_cache_6.py

```

1  import streamlit as st
2  import pandas as pd
3  import time
4
5  st.title("Przykład: buforowanie danych")
6
7  st.write("""
8  Bez '@st.cache_data' dane byłyby wczytywane od nowa przy każdym
9  kliknięciu lub zmianie widżetu. Z dekoratorem – wynik jest
10 zapamiętywany i ponownie używany.
11 """)
12
13 @st.cache_data
14 def wczytaj_dane(uploaded_file):
15     time.sleep(2) # symulacja długiego wczytywania
16     return pd.read_csv(uploaded_file)
17
18 uploaded = st.file_uploader("Wgraj plik CSV")
19
20 if uploaded is not None:
21     st.write("Pierwsze wczytanie zajmie 2 sekundy...")
22     with st.spinner("Wczytuje..."):
23         df = wczytaj_dane(uploaded)
24     st.success("Dane wczytane! Kolejne uruchomienia będą natychmiastowe")
25     st.metric("Wiersze", df.shape[0])
26     st.metric("Kolumny", df.shape[1])
27     st.dataframe(df.head())
28 else:
29     st.info("Wgraj plik CSV, aby zobaczyć działanie cache.")
30

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

waweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_session_5.py
waweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_cache_6.py

```

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>
 Network URL: <http://192.168.1.168:8501>

For better performance, install the Watchdog module:

```

$ xcode-select --install
$ pip install watchdog

```

Przykład: buforowanie danych

Bez `@st.cache_data` dane byłyby wczytywane od nowa przy każdym kliknięciu lub zmianie widżetu. Z dekoratorem – wynik jest zapamiętywany i ponownie używany.

Wgraj plik CSV



Drag and drop file here

Limit 200MB per file

Browse files

Wgraj plik CSV, aby zobaczyć działanie cache.

Wiele kolumn — elastyczny układ

```
import streamlit as st

# Równe kolumny
col1, col2, col3 = st.columns(3)

# Kolumny z różnymi szerokościami (proporcje)
col_szer, col_wask = st.columns([3, 1])

with col_szer:
    st.write("Szeroka kolumna - np. wykres")
with col_wask:
    st.write("Wąska - np. filtr")
```

- Lista liczb w `st.columns([3, 1])` określa proporcje.
- Można tworzyć układy: wykres + panel kontrolny, mapa + tabela itp.

Uruchom app_columns_7.py

```

app_columns_7.py > ...
1  import streamlit as st
2  import pandas as pd
3  import numpy as np
4
5  st.title("Układ z kolumnami")
6
7  # Kolumny z proporcjami 3:1
8  col_wykres, col_filtr = st.columns([3, 1])
9
10 with col_filtr:
11     st.subheader("Filtry")
12     n = st.slider("Liczba punktów", 10, 200, 50)
13     srednia = st.slider("Średnia", -3.0, 3.0, 0.0)
14     odch = st.slider("Odch. std.", 0.1, 3.0, 1.0)
15
16 with col_wykres:
17     st.subheader("Wykres rozrzutu")
18     dane = pd.DataFrame({
19         "x": np.random.normal(srednia, odch, n),
20         "y": np.random.normal(srednia, odch, n)
21     })
22     st.scatter_chart(dane)
23
24 # Drugi przykład: równe kolumny
25 st.divider()
26 st.subheader("Równe kolumny (3 x 1/3)")
27
28 c1, c2, c3 = st.columns(3)
29 with c1:
30     st.info("Kolumna 1\ntekst, metryki, filtry")
31 with c2:

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

ewaweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_cache_6.py
ewaweychert@Pawes-MacBook-Air 5_streamlit % streamlit run app_columns_7.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.168:8501

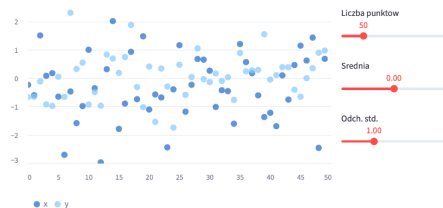
For better performance, install the Watchdog module:

$ xcode-select --install
$ pip install watchdog

```

Układ z kolumnami

Wykres rozrzutu



Filtry



Równe kolumny (3 x 1/3)

Kolumna 1 Tekst, metryki, filtry

Kolumna 2 Wykresy, tabele

Kolumna 3 Opisy, linki

Zaawansowane opcje layoutu

Wiele stron — folder pages/

Struktura folderów:

```
app_multipage/  
  app.py  
  pages/  
    1_Dane.py  
    2_Wykresy.py  
    3_0_aplikacji.py
```

- Streamlit automatycznie wykrywa pliki w folderze pages/.
- Każdy plik staje się osobną stroną w menu bocznym.
- Numerowanie plików (1_, 2_) kontroluje kolejność w menu.
- Idealne dla większych aplikacji z wieloma sekcjami.

streamlit run app_multipage/app.py

```

Welcome  app.py x
app_multipage > app.py
1  import streamlit as st
2
3  st.set_page_config(
4      page_title="Moja Aplikacja",
5      layout="wide"
6  )
7
8  st.title("Strona glowna")
9  st.write("Wybierz sekcje z menu po lewej stronie.")
10
11  st.markdown("""
12  ### Dostepne sekcje:
13  - **Dane** – wczytaj i przejrzyj swoj zbior danych
14  - **Wykresy** – wizualizacje i wykresy
15  - **O aplikacjach** – informacje o projekcie
16  """)
17
18  st.info("Ta aplikacja demonstruje uklad wielostronicowy w Streamlit.")
19

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

ewaweychert@Pawes-MacBook-Air: ~ % streamlit run app_multipage/app.py
ewaweychert@Pawes-MacBook-Air: ~ % streamlit run app_multipage/app.py

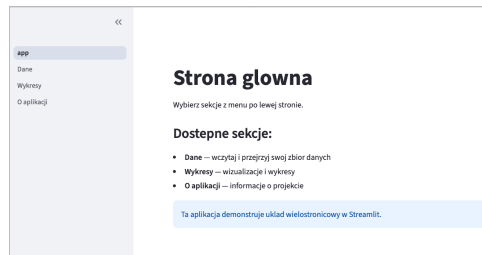
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.168:8501

For better performance, install the Watchdog module:

$ code-select --install
$ pip install watchdog

```



Przykład strony pages/1_Dane.py

```
import streamlit as st
import pandas as pd

st.title("Dane")
st.write("Ta strona pokazuje wczytane dane.")

uploaded = st.file_uploader("Wgraj plik CSV")
if uploaded:
    df = pd.read_csv(uploaded)
    st.dataframe(df)
```

- Każda strona to osobny plik .py — piszesz go tak samo jak zwykłą aplikację Streamlit.
- Stan sesji (`st.session_state`) jest współdzielony między stronami.

Własny motyw — config.toml

Struktura folderów:

```
moja_aplikacja/  
  app.py  
  .streamlit/  
    config.toml
```

Zawartość pliku config.toml:

```
[theme]  
primaryColor = "#991e1e"  
backgroundColor = "#ffffff"  
secondaryBackgroundColor = "#f0f2f6"  
textColor = "#262730"  
font = "sans serif"
```

- primaryColor — kolor przycisków i akcentów.
- Można dopasować kolory do identyfikacji wizualnej uczelni lub firmy.

Ograniczenia config.toml

Problem: `secondaryBackgroundColor` zmienia kolor paska bocznego tylko na **jasne odcienie**. Dla ciemnego czerwonego tekst staje się nieczytelny.

Rozwiązanie — CSS bezpośrednio w `app.py`:

```
st.markdown("""
<style>
[data-testid="stSidebar"] {
background-color: #991e1e;
}
[data-testid="stSidebar"] * {
color: white !important;
}
</style>
""", unsafe_allow_html=True)
```

- `data-testid="stSidebar"` — wskazuje pasek boczny Streamlit.
- `color: white !important` — wymusza biały kolor tekstu na ciemnym tle.
- `unsafe_allow_html=True` — pozwala Streamlit wczytać kod HTML/CSS.

Ograniczenia config.toml

Problem: `secondaryBackgroundColor` zmienia kolor paska bocznego tylko na **jasne odcienie**. Dla ciemnego czerwonego tekst staje się nieczytelny.

Rozwiązanie — CSS bezpośrednio w `app.py`:

```
st.markdown("""
    <style>
      [data-testid="stSidebar"] {
        background-color: #991e1e;
      }
      [data-testid="stSidebar"] * {
        color: white !important;
      }
    </style>
    """, unsafe_allow_html=True)
```

- `data-testid="stSidebar"` — wskazuje pasek boczny Streamlit.
- `color: white !important` — wymusza biały kolor tekstu (czytelny na ciemnym tle).
- `unsafe_allow_html=True` — pozwala Streamlit wczytać kod HTML/CSS.

Co mozna zmienic gdzie? — podsumowanie

Co zmieniasz	Gdzie	Trudnosc
Kolor przyciskow i linkow	<code>config.toml</code>	Latwe
Kolor tla glownego	<code>config.toml</code>	Latwe
Kolor tekstu	<code>config.toml</code>	Latwe
Czcionka	<code>config.toml</code>	Latwe
Max rozmiar pliku	<code>config.toml</code>	Latwe
Kolor paska bocznego (jasny)	<code>config.toml</code>	Latwe
Kolor paska bocznego (ciemny)	CSS w <code>app.py</code>	Srednie
Logo wydzialu	kod w <code>app.py</code>	Latwe
Uklad strony	kod w <code>app.py</code>	Latwe

Wskazowka

Zaczniij od `config.toml` — to wystarczy dla wiekszosci projektow studenckich.
CSS jest potrzebny tylko gdy chcesz ciemny pasek boczny.

Jak dodac logo wydzialu?

Krok 1 — wgraj plik wne_uw.png do folderu obok app.py:

```
moja_aplikacja/  
  app.py  
  wne_uw.png      (tutaj)  
  .streamlit/  
    config.toml
```

Krok 2 — dodaj do app.py:

```
import os  
  
# Logo w pasku bocznym  
st.sidebar.image(  
    os.path.join(os.path.dirname(__file__), "wne_uw.png"),  
    width=180  
)  
  
# Logo na stronie glownej (wysrodkowane)  
col1, col2, col3 = st.columns([1, 2, 1])  
with col2:  
    st.image(  
        os.path.join(os.path.dirname(__file__), "wne_uw.png"),  
        width=300  
    )
```

- `os.path.dirname(__file__)` — zwraca folder, w ktorym jest `app.py`.
- Dzieki temu sciezka dziala na kazdym komputerze.

Opcje wdrożenia aplikacji Streamlit

Opcje wdrożenia aplikacji Streamlit

- **Streamlit Community Cloud** (bezpłatne)
 - Najprostsze rozwiązanie — tylko GitHub i przeglądarka.
 - Idealne do projektów studenckich i demonstracji.
- **Własny serwer / VPS**
 - Pełna kontrola, wymaga konfiguracji serwera.
- **Platformy chmurowe** (AWS, GCP, Azure)
 - Dla aplikacji produkcyjnych z dużym ruchem.

Streamlit Community Cloud — jak to działa

- ❶ Utwórz konto na `github.com` (jeśli nie masz).
- ❷ Utwórz nowe repozytorium i wgraj do niego:
 - plik `app.py`,
 - plik `requirements.txt`.
- ❸ Wejdź na `share.streamlit.io` i zaloguj się przez GitHub.
- ❹ Kliknij „**New app**” i wybierz swoje repozytorium.
- ❺ Streamlit automatycznie zainstaluje zależności i uruchomi aplikację.
- ❻ Otrzymasz publiczny link — np. `twoja-app.streamlit.app`.

Plik requirements.txt

Przykład pliku requirements.txt:

```
streamlit  
pandas  
matplotlib  
seaborn  
numpy  
scikit-learn
```

- Zawiera listę bibliotek Pythona potrzebnych do uruchomienia aplikacji.
- Streamlit Cloud automatycznie instaluje wszystkie wymienione pakiety.
- Można podać konkretne wersje:

```
streamlit==1.35.0  
pandas==2.2.1
```

- Zalecane dla stabilności — zapobiega problemom z nowszymi wersjami.

Struktura projektu do wdrożenia

```
moj_projekt/  
  app.py  
  requirements.txt  
  README.md  
  data/  
    dane.csv  
  pages/  
    1_EDA.py  
    2_Wyniki.py  
  .streamlit/  
    config.toml
```

- Prosta, przejrzysta struktura ułatwia wdrożenie i ocenę projektu.
- Cały folder wgraj do repozytorium GitHub.

Czym jest GitHub i Streamlit Cloud?

- **GitHub** — serwis do przechowywania kodu w internecie.
 - Twój kod jest zapisany w **repozytorium** (repo).
 - Można udostępniać kod innym i współpracować.
 - Streamlit Cloud czyta kod właśnie z GitHub.
- **Streamlit Community Cloud** — bezpłatny hosting aplikacji Streamlit.
 - Twoja aplikacja działa w internecie 24/7.
 - Każdy może ją otworzyć przez link.
 - Aktualizuje się automatycznie po zmianie kodu.

Jak to działa?

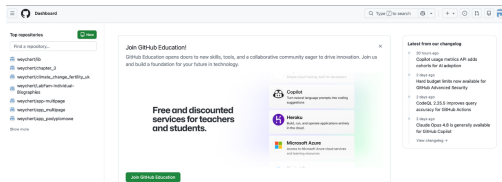
Piszesz kod na komputerze → wysyłasz na GitHub → Streamlit Cloud pobiera kod → aplikacja działa w internecie.

Czym jest GitHub i Streamlit Cloud?

The screenshot shows the GitHub profile of a user named 'weychert'. The profile includes a circular avatar with a blue and white geometric design, the username 'weychert', an 'Edit profile' button, and a note 'At 1 follower · 1 following'. The 'Popular repositories' section lists three repositories: 'LabFam-Individual-Biographies' (R, 2 stars), 'app_podyplomowe' (Python), and 'app_multipage' (Python). The '32 contributions in the last year' section shows a calendar grid with green squares indicating contributions. The 'Contribution activity' section shows a commit history for the repository 'weychert/app_podyplomowe'.

Stwórz konto lub się zaloguj: <https://github.com>

Krok 1 — GitHub Dashboard



- Zaloguj się na github.com.
- Widzisz swoje repozytoria po lewej stronie.
- Kliknij zielony przycisk **New** aby stworzyć nowe repozytorium.

Krok 2 — Stworz nowe repozytorium

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 **General**

Owner * weychert / Repository name *

app-multipage is available.

Great repository names are short and memorable. How about [friendly-bassoon](#)?

Description

0 / 350 characters

2 **Configuration**

Choose visibility * Public
Choose who can see and commit to this repository

Add README off
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
Licenses explain how others can use your code. [About licenses](#)

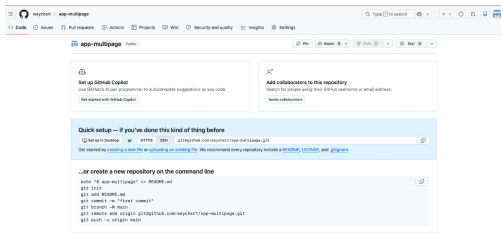
[Creating repository...](#)

- Wpisz nazwe repozytorium np. app-multipage.
- Zaznacz **Public** (wymagane przez Streamlit Cloud).
- **Nie zaznaczaj** Add README — pliki wyslemy sami z terminala.
- Kliknij **Create repository**.

Uwaga

Nazwa repo musi byc **identyczna** z nazwa uzyta w komendzie `git remote add origin`.

Krok 3 — Wyslij pliki z terminala



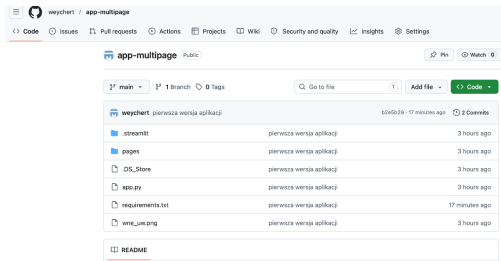
Wpisz w terminalu po kolei:

```
cd ../app_multipage
```

```
git init
git add .
git commit -m "pierwsza wersja"
git remote add origin
    git@github.com:NAZWA/app-multipage.git
git branch -M main
git push -u origin main
```

Efekt: pliki pojawiają się na GitHub — widac `app.py`, `pages/`, `requirements.txt`.

Krok 4 — Pliki sa na GitHub!



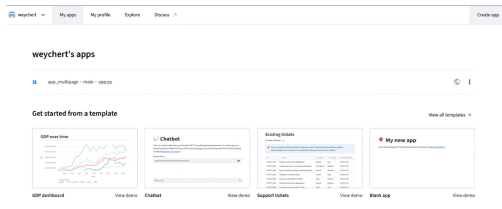
Sprawdz na GitHub czy widzisz:

- folder `.streamlit/`
- folder `pages/`
- plik `app.py`
- plik `requirements.txt`
- plik `wne_uw.png`

Sukces!

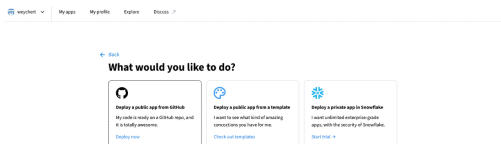
Jesli widzisz te pliki — mozesz przejsc do wdrozenia na Streamlit Cloud.

Krok 5 — Streamlit Cloud



- Wejdz na `share.streamlit.io`.
- Zaloguj sie przez GitHub.
- Kliknij **Create app**.
- Wybierz opcje: **Deploy a public app from GitHub**.

Krok 6 — Wybierz repozytorium



Wypełnij formularz:

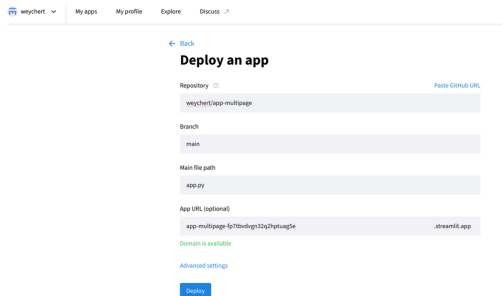
- **Repository:**
weychert/app-multipage
- **Branch:** main
- **Main file path:** app.py

Kliknij **Deploy!**

Czesty błąd

Branch musi być main a nie master.
Plik musi być app.py a nie
streamlit_app.py.

Krok 7 — Aplikacja się buduje



The screenshot shows the 'Deploy an app' interface on Streamlit Cloud. At the top, there's a navigation bar with 'weychert' and links for 'My apps', 'My profile', 'Explore', and 'Discuss'. Below this, a 'Back' link is visible. The main heading is 'Deploy an app'. The form includes several input fields: 'Repository' (containing 'weychert/app-multipage' with a 'Paste GitHub URL' link), 'Branch' (containing 'main'), 'Main file path' (containing 'app.py'), and 'App URL (optional)' (containing 'app-multipage-4p7t0vdiq312q3hptuag5e' with a '.streamlit.app' suffix). A green status message 'Domain is available' is shown below the App URL field. There is also a link for 'Advanced settings' and a blue 'Deploy' button at the bottom.

- Streamlit Cloud pobiera kod z GitHub.
- Instaluje pakiety z `requirements.txt`.
- Uruchamia aplikację.
- Czekaj ok. 2–3 minuty.

Your app is in the oven

Ten ekran oznacza że aplikacja się buduje — to normalne! Poczekaj chwilę.

Krok 8 — Aplikacja działa!



- Aplikacja działa w internecie!
- Widoczne menu boczne z zakładkami.
- Logo WNE widoczne.
- Kolory WNE aktywne.

Gotowe!

Kliknij **Copy link** i wyślij link prowadzącym lub znajomym. Aplikacja działa 24/7.

<https://app-multipage-fp7tbvdvgn32q2hptuag5e.streamlit.app>

Jak aktualizować aplikację po zmianach?

Gdy zmienisz coś w kodzie na komputerze:

```
cd ~/Desktop/.../app_multipage  
  
git add .  
git commit -m "opis zmiany np. dodano nowy wykres"  
git push
```

Streamlit Cloud automatycznie wykryje zmianę i zaktualizuje aplikację w ciągu 1–2 minut.

Sytuacja	Co robić
Pierwsza publikacja	git init, add, commit, push
Kolejne zmiany	tylko git add, commit, push
Sprawdzenie statusu	git status
Historia commitów	git log

Token zamiast hasła

GitHub wymaga **tokenu** zamiast hasła przy git push.

Stwórz go na: [github.com](#) → Settings → Developer settings → Personal access tokens.

Podsumowanie — ścieżka wdrożenia

Krok	Co robisz	Gdzie
1	Piszesz aplikację	VS Code, terminal
2	Zakładasz konto	github.com
3	Tworzysz repozytorium	github.com
4	<code>git init</code>	terminal
5	<code>git add .</code>	terminal
6	<code>git commit</code>	terminal
7	<code>git push</code>	terminal
8	Lączysz z Streamlit Cloud	share.streamlit.io
9	Klikasz Deploy	share.streamlit.io
10	Aplikacja działa w sieci	Twój link

Wskazówka dla projektu

Wgraj aplikację na GitHub przed prezentacją.
Prowadzący będzie mógł otworzyć ją przez link
bez instalowania czegokolwiek na swoim komputerze.

Dobre praktyki przy tworzeniu aplikacji Streamlit

- **Struktura kodu**

- Używaj funkcji pomocniczych zamiast pisać wszystko w jednym bloku.
- Rozdzielaj logikę danych od warstwy UI.
- Duże aplikacje dziel na strony (pages/).

- **Wydajność**

- Używaj `@st.cache_data` dla operacji wczytywania danych.
- Unikaj ciężkich obliczeń bez buforowania.

- **Użyteczność**

- Dodawaj tytuły, podpisy i opisy do każdej sekcji.
- Informuj użytkownika o stanie aplikacji (`st.spinner`, `st.success`).
- Obsługuj przypadek braku danych (brak wgranego pliku).

Obsługa błędów w aplikacji

- Zawsze sprawdzaj, czy plik został wgrany przed próbą jego odczytu:

Dobra praktyka

```
uploaded = st.file_uploader("Wgraj plik CSV")
if uploaded is not None:
    df = pd.read_csv(uploaded)
    # reszta kodu...
else:
    st.info("Wgraj plik CSV, aby rozpocząć.")
```

- Używaj try / except dla operacji mogących się nie powieść.
- Wyświetlaj zrozumiałe komunikaty błędów zamiast technicznych wyjątków.

Podsumowanie

- **Zaawansowane widgety:**
 - `st.expander`, `st.metric`, `st.progress`, `st.spinner`, `st.form`, `st.session_state`, `@st.cache_data`.
- **Zaawansowany layout:**
 - Proporcjonalne kolumny, wiele stron (`pages/`), własny motyw (`config.toml`).
- **Wdrożenie:**
 - Streamlit Community Cloud — najprościej, bezpłatnie.
 - Docker — przenośność i powtarzalność środowiska.
 - `requirements.txt` — klucz do poprawnego wdrożenia.
- **Dobre praktyki:**
 - Funkcje pomocnicze, buforowanie, obsługa błędów.

Literatura i zasoby

- <https://streamlit.io/cloud>
- <https://docs.streamlit.io>
- <https://docs.streamlit.io/develop/concepts/multipage-apps>
- <https://docs.docker.com/get-started/>
- <https://github.com/streamlit/streamlit>